

PoolParty: streamlined design of DNA sequence libraries in Python

Zhihan Liu¹, Aidan Cordero¹, Justin B. Kinney^{1*}

^{1*}Simons Center for Quantitative Biology, Cold Spring Harbor
Laboratory, 1 Bungtown Rd., Cold Spring Harbor, New York, 11724,
United States.

*Corresponding author(s). E-mail(s): jkinney@cshl.edu;
Contributing authors: zhliu@cshl.edu; aicorder@cshl.edu;

Abstract

Background: Computationally designed DNA sequence libraries are essential components of massively parallel reporter assays (MPRAs), deep mutational scanning (DMS) experiments, and other multiplex assays of variant effect (MAVEs). They are also increasingly used in silico to analyze genomic AI models. Designing these libraries, however, remains tedious and error-prone due to the lack of purpose-built software.

Results: Here we describe PoolParty, a Python package that streamlines the design of complex oligo pools using a simple but flexible API. In PoolParty, each library is represented by a computational graph that can be specified in just a few lines of code. Over 50 built-in operations cover nucleotide- and codon-level mutagenesis, motif insertion, barcode generation, and more. PoolParty automatically generates informative names for each sequence and provides “design cards” detailing how each sequence was generated. Visualization methods let users quickly audit library content and inspect the underlying graph. PoolParty thus transforms oligo pool design from a tedious task requiring custom functions and scripts into a structured, transparent, and reproducible process.

Conclusions: PoolParty should substantially ease the design of DMS, MPRA, and other multiplex assay libraries by replacing ad hoc scripts with a more structured and reproducible framework. We expect design cards to be especially useful for in silico experiments on genomic AI models, and PoolParty can be extended to support new assay types and analysis strategies as they emerge.

Keywords: sequence design, library design, massively parallel reporter assay, deep mutational scanning, multiplex assay of variant effect, genomic AI, in silico experiments, Python software, directed acyclic graph

1 Background

Designed libraries of DNA sequences (oligonucleotide pools) have become essential tools in functional genomics. In massively parallel reporter assays (MPRAs), complex libraries of DNA sequences are routinely used to measure the regulatory activities of tens of thousands of candidate regulatory elements or their variants in a single experiment [1–13]. In deep mutational scanning (DMS) experiments, coding sequences that have been either systematically or randomly mutagenized are commonly used to map protein fitness landscapes [14–20]. Oligo pools can also be used in high-throughput biochemical assays, such as measurements of transcription factor specificity [21–27]. These are all examples of multiplex assays of variant effect (MAVEs), a large and expanding family of high-throughput methods that quantitatively link sequence to function at scale [28–30]. Designed sequence libraries are also used beyond the wet lab: they are increasingly employed in silico to probe the behavior of genomic AI models [31–35] and to augment genomic AI training data [36].

Computationally designing these libraries is straightforward in principle, but in practice it is often tedious and prone to error. Typical applications require oligo pools comprising multiple types of variant sequences. A DMS library, for example, might combine exhaustive single-amino-acid substitutions, sampled higher-order mutants, multiple copies of wild-type and control sequences, and barcodes—each generated by a different procedure and subject to different coverage requirements. An MPRA library might tile multiple transcription factor binding sites across a regulatory element in all possible permutations and orientations, then assign each variant multiple

barcodes. Coordinating this kind of combinatorial logic in a one-off script quickly becomes unwieldy, and often produces errors that are hard to catch.

Existing tools can help with parts of this process, but none provide a unified framework for library design. General-purpose sequence toolkits [37–40] can manipulate individual sequences but have no notion of a variant library as a structured object. Consequently, users must write custom scripts to handle the combinatorial logic themselves. Several assay-specific tools automate library design for particular experimental formats. For example, VaLiAnT [41] generates exhaustive single-position variant libraries for DMS and saturation genome editing, including single-nucleotide substitutions, alanine scans, and in-frame deletions. For MPRA, MPRAinator [42] provides modules for motif placement, SNP design, and negative control generation [see also 43]. However, each of these tools is tied to a specific assay type and does not support combinatorial designs or mixed mutagenesis strategies. While some annotate each variant with the specific mutations it carries, none record the broader design parameters governing how each sequence was generated—parameters that can serve directly as covariates in downstream analyses.

Here we present PoolParty, a Python package that enables the streamlined design of complex DNA sequence libraries. In PoolParty, libraries are represented as objects called *Pools*, while individual steps in the sequence generation process are called *Operations*. Using a concise and transparent syntax, users chain together Operations into a directed acyclic graph (DAG)—an approach inspired by deep learning frameworks—that yields the desired Pool. The DAG describes the high-level logic used to generate sequences, while the procedural details and bookkeeping are handled internally. Importantly, no sequences are generated until the user requests them, so users can explore and test multiple design options without triggering expensive computations. When sequences are generated, each is automatically assigned an informative name and paired with a *design card* that records the specific procedures used to construct it.

139 We demonstrate PoolParty through three example applications: a DMS library for
140 protein GB1, an MPRA library probing transcriptional regulatory grammar, and an
141 in silico experiment characterizing SpliceAI [44], in which design cards provide the
142 covariates for surrogate modeling.
143

144

145

146 2 Implementation

147

148

149 2.1 Pools and Operations

150

151 PoolParty is built around two core abstractions: Pools and Operations. Pools represent
152 collections of sequences—either the final library that the user wishes to generate, or
153 intermediate sets of sequences out of which the final library is constructed. Operations
154 represent the steps used to create Pools. When designing a library, users chain together
155 multiple Operations into a DAG that yields the desired Pool, called the *root Pool*.
156 Sequences are then generated on demand from the root Pool. By delaying sequence
157 generation until it is explicitly requested, users can test many different library designs
158 before committing to generating the full library. For example, users can inspect a few
159 sampled sequences from each one of multiple proposed designs.
160

161 PoolParty provides over 50 built-in Operations that users can choose from when
162 constructing DAGs. Each Operation takes zero or more Pools as input and returns
163 exactly one Pool as output. It is useful to think of Operations as falling into
164 four functional categories (Fig. 1A): source, transformation, composition, and state
165 Operations. *Source Operations* create the initial Pools from which all downstream
166 Pools are constructed. The Pools they create may represent user-specified sequences,
167 DNA sequences generated using position weight matrices (PWMs) or IUPAC motifs,
168 random k-mers, barcodes, and so on. *Transformation Operations* modify sequence
169 content, e.g., generating sequence variants through point mutations, insertions, dele-
170 tions, recombination, or shuffling. Specialized codon-aware Operations are provided for
171

172

173

174

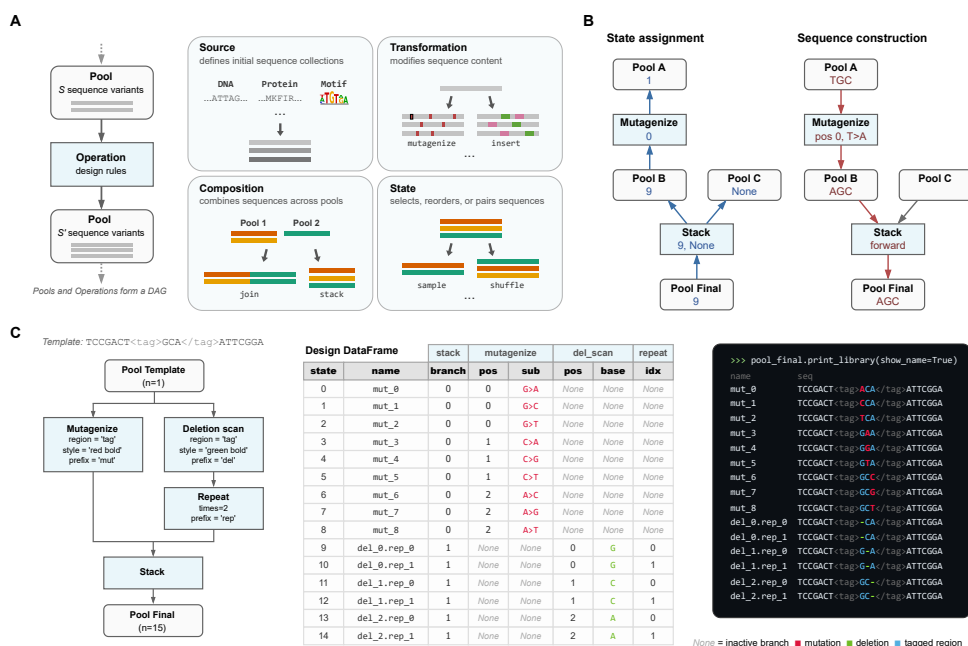


Fig. 1 PoolParty builds libraries from Pools and Operations. (A) Pools represent collections of sequences, and Operations generate new Pools based on their inputs (left). Users chain Operations into a directed acyclic graph (DAG) that specifies the desired library. Operations fall into four functional categories: source, transformation, composition, and state (right). (B) Sequence generation proceeds in two steps: state assignment and sequence construction. For each desired sequence, PoolParty passes to the root Pool a number (called the *state*) that specifies which sequence to generate. During state assignment (left), this state is propagated backward through the DAG, resolving the internal state of every Operation along the way. During sequence construction (right), each Operation generates and combines sequence fragments according to its assigned state, producing the final sequence at the root Pool. When multiple Pools are combined by *stack*, only one parent is active for a given state; inactive branches receive no state (*None*) and are shown as grey arrows. (C) Example workflow. The template sequence contains a user-specified region (called “tag”) that is targeted for both mutagenesis and deletion scanning. Left: the DAG specifying the sequence library. Center: the design card DataFrame returned by the DAG. Right: styled sequence output, with colors indicating mutations (red), deletions (green), and the tagged region (blue).

transformations that operate on open reading frames. *Composition Operations* combine sequences from multiple parent Pools, either by concatenating parent sequences end-to-end (the *join* Operation) or by merging multiple Pools into one Pool (the *stack* Operation). *State Operations* affect which sequences are in a Pool and how they are ordered, and include methods to select, reorder, filter, and replicate sequences.

2.2 Sequence generation

To generate a sequence library, PoolParty iterates through a series of *states*. Each state is an integer that, together with a random seed, uniquely determines a sequence. The generation of each individual sequence proceeds in two steps, state assignment and sequence construction (Fig. 1B), a process that begins when the corresponding state is passed to the root Pool. State assignment involves a *backward pass* through the DAG. Each Pool passes the state it receives to its upstream Operation. Each Operation then decomposes this state into an *internal state* (which it keeps) and zero or more *component states*, each of which is passed to a distinct upstream Pool. Sequence construction then occurs during a *forward pass* through the DAG: each Operation constructs a sequence based on its internal state and the sequences received from the upstream Pools, then passes this sequence to the downstream Pool. The final constructed sequence is then received from the root Pool and recorded.

2.3 Operation modes

The specific way that each Operation interprets its internal state is determined by its *mode*.

- In *sequential mode*, the Operation exhaustively enumerates a finite set of sequence designs. For example, when `mutagenize` is instructed to generate single-nucleotide mutations in sequential mode, it yields one variant for each possible character change at each target position. Its internal state specifies which mutation to apply at which position.
- In *random mode*, the Operation samples sequence designs stochastically. For example, when `mutagenize` operates in random mode, it randomly selects which positions to mutate and which mutations to introduce. Its internal state, combined with a master seed, determines the specific random draws, ensuring reproducibility.

- In *fixed mode*, there is no internal state. Rather, output sequences are uniquely determined by the input sequences. One example is `join`, which concatenates its input sequences without modification.

2.4 StateTracker

The way each Operation decomposes the state it receives during the backward pass is designed to reverse the way it constructs sequences during the forward pass. For most Operations, the set of possible output states is the Cartesian product of the possible internal states and the possible states of each input Pool. In such cases, the Operation uses mixed-radix decomposition to split the received state into its component states. The `stack` Operation is different: its output states are the disjoint union of its input Pool states. The `stack` Operation therefore uses the state it receives to select which input Pool to use, and to instruct this Pool (but not the other Pools) which sequence to return. Keeping track of these rules across a complex DAG requires specialized bookkeeping. We therefore developed a back-end package called StateTracker that automates state composition and decomposition during forward and backward passes through the DAG.

2.5 Sequence regions

One may want to perform different Operations on different parts of a sequence. To facilitate this, PoolParty lets users mark regions of interest within sequences using XML-style tags. PoolParty supports both opening/closing tag pairs (e.g., `AAA<cre>CCCCGGG</cre>TTT`) and self-closing tags (e.g., `ACGT<ins/>ACGT`). Operations can then be instructed to act only on a specified region, as illustrated in Fig. 1C. Tags persist through the DAG and remain valid even when upstream Operations change the content of a region. Multiple Operations can thus target modifications to the same region in series.

323 2.6 Sequence metadata

324

325 It is often useful to record how each sequence within a library was constructed.

326

327 PoolParty provides three complementary ways to do this (Fig. 1C).

328

329 • *Design cards* record the specific changes applied to each sequence by each Operation.

330

331 Users specify which Operations and Operation-specific variables to track. As each

332

332 sequence is generated, the root Pool returns it together with a design card that

333

334 records the tracked variable values for that sequence. PoolParty compiles these into

335

336 a DataFrame in which each row contains a generated sequence and its design card.

337

338 • *Sequence names* summarize at a glance how each sequence was constructed. Users

339

339 assign a prefix to each Operation they wish to track. PoolParty then pairs each

340

341 prefix with the corresponding Operation's internal state to form a token (e.g.,

342

342 `mut_03`), and concatenates these tokens to form the sequence's name. For exam-

343

344 ple, `wt.mut_03.bc_01` might denote a wild-type sequence with mutation variant

345

346 number 3 and barcode number 1.

347

348 • *Sequence styling* allows sequences to be annotated with colors and formatted text

349

349 that reflects each one's construction history. For example, `mutagenize` can be

350

351 instructed to mark the positions it mutates in bold yellow text, making these posi-

352

352 tions stand out from the rest of the sequence. This formatting is accomplished using

353

354 ANSI escape codes, so that output renders correctly in command-line terminals,

355

356 Jupyter notebooks, and other environments that support basic text formatting.

357

358 These styles combine in a natural manner as sequences propagate through the

359

359 DAG. For example, a mutated position can be underlined even if it occurs within

360

361 a sequence region that is already colored blue.

362

363

364 2.7 Implementation and extensibility

365

366 PoolParty requires Python 3.10 or later, is designed to have minimal dependencies,

367

367 and is distributed via PyPI. Documentation, tutorials, and example notebooks are

368

available on the ReadTheDocs and GitHub project websites. PoolParty is also extensible: users can create new Operation types by subclassing the Operation base class and implementing a few required methods. Custom Operations automatically inherit support for state tracking, design card generation, and visualization, allowing researchers to add domain-specific functionality without reimplementing core infrastructure.

2.8 SpliceAI surrogate analysis

This section provides technical details for the surrogate modeling analysis presented in Results. Using the human 5' splice site (5'ss) position weight matrix [45], we sampled 9-mers representing cryptic 5'ss sequences and scored them with MaxEntScan [46]. Sequences were binned into 100 MaxEntScan score (hereafter, strength) quantiles, and 20 sequences were drawn from each bin (2,000 total). Each 9-mer was inserted at 100 positions flanking the canonical 5'ss of SMN2 exon 7 and paired with a matched control in which the GT dinucleotide was disrupted (T>A at +2).

We predicted SpliceAI 5'ss probabilities at the canonical site using the five-model ensemble of Jaganathan et al. [44] with ± 5 kb of genomic context (GRCh38). For each library-control pair, we computed the difference in logit-transformed predictions at the canonical 5'ss. These per-sequence values were averaged within each cell of the resulting 100×100 position-by-strength grid.

We fit generalized additive models (GAMs) to the cell means using pyGAM [47], with smooth terms for position, strength, and their interaction. Models were weighted by the inverse variance of each cell mean. Exonic and intronic regions were modeled separately. Model performance was evaluated by checkerboard cross-validation of the grid.

415 2.9 Use of large language models

416

417 Large language models (Claude Opus 4.5 and 4.6, Anthropic) were used to assist
418 with code development, debugging, documentation writing, and copy-editing of the
419 manuscript. The authors are responsible for all final code and text.
420
421

422

423 3 Results

424

425

426 3.1 Example 1: DMS library for protein GB1

427

428 As a first example, we use PoolParty to reconstruct and extend the library used in
429 a DMS experiment on protein GB1 by Olson et al. [17], adding random higher-order
430 mutants and wild-type replicates to the original design. Their library contained the
431 wild-type GB1 coding sequence, all single amino acid substitutions (excluding the
432 start codon), and nearly all possible pairs of amino acid substitutions. This library is
433 substantially larger than most DMS libraries, containing over half a million variants.
434
435 Its inclusion of so many pairwise variants has helped it become a benchmark dataset
436 in the quantitative modeling of protein sequence-function relationships [48–51].
437
438

439 Fig. 2A illustrates the DAG used to construct this library; the corresponding
440 Python code is shown in Fig. 2B. First, we use the `from_seq` Operation to define a
441 Pool named `orf_pool` that represents the wild-type GB1 coding sequence. This Pool
442 is then passed to four separate Operations that define the four library components.
443 The first is a `repeat` Operation that produces 100 copies of the wild-type sequence.
444 The second and third are `mutagenize_orf` Operations running in sequential mode:
445 one generates all 1,045 single amino acid substitutions, and the other generates all
446 536,085 pairwise amino acid substitutions. The fourth is a `mutagenize_orf` Oper-
447 ation running in random mode, which mutates each amino acid position with 10%
448 probability to generate 10,000 higher-order variants. We note that `mutagenize_orf`
449 supports multiple codon mutation types; this example uses `missense_only_first`,
450
451
452
453
454
455
456
457
458
459
460

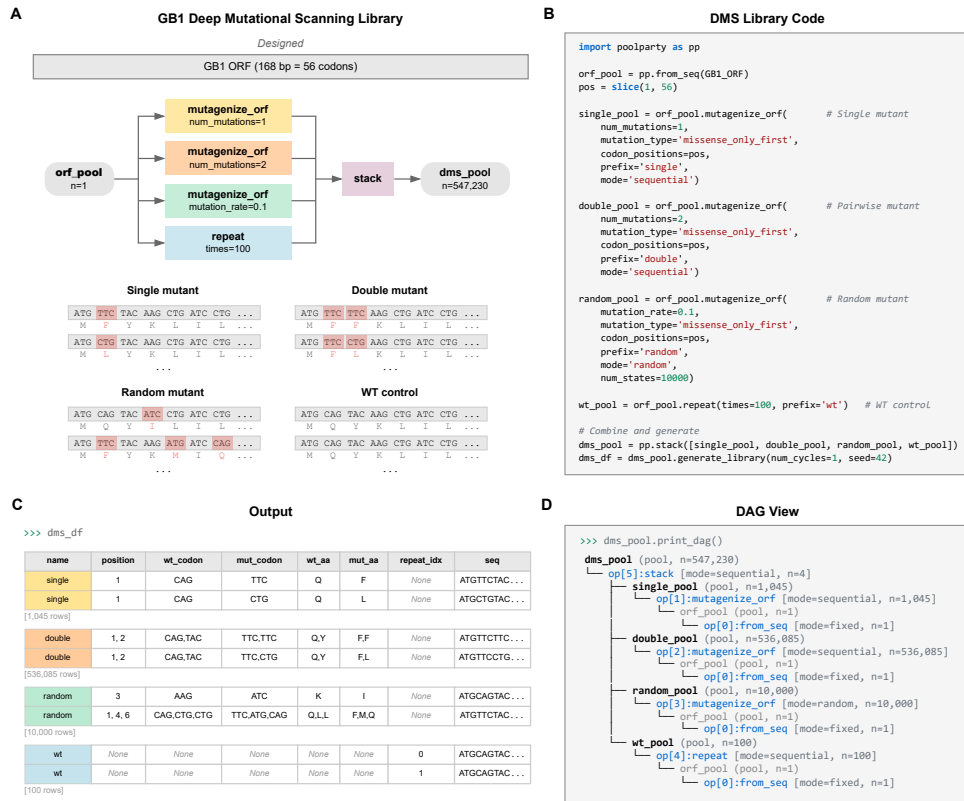


Fig. 2 A deep mutational scanning (DMS) library for protein GB1. (A) DAG and representative sequences for each of the four library components: replicates of the wild-type sequence, all single amino acid substitutions, all pairwise amino acid substitutions, and random higher-order mutants. Mutated codons are highlighted with amino acid translations shown beneath. **(B)** Python code implementing the DAG. **(C)** Design cards reporting the library component, mutation positions, and amino acid changes for representative variants. **(D)** DAG structure displayed by `print_dag()`.

which selects the most abundant codon for each amino acid substitution. These four Pools are then merged by `stack` into the root Pool, yielding a library of 547,230 sequences in total. The design card for each variant (Fig. 2C) reports which component it belongs to, the specific mutations it contains, and the replicate index (for wild-type copies). Users can also inspect the DAG structure by calling the `print_dag` method of the root Pool (Fig. 2D).

507 3.2 Example 2: MPRA library for regulatory grammar

508
509 MPRA libraries used to study regulatory grammar often vary the positions and ori-
510 entations of transcription factor binding sites (TFBSs) [3, 5, 52–56]. For example,
511 Georgakopoulos-Soares et al. [56] studied the impact of order and orientation on
512 gene expression by testing over 200,000 sequences containing all pairwise and triplet
513 arrangements of 18 TFBSs. Designing such libraries requires substantially more com-
514 plex combinatorics than standard DMS libraries do. PoolParty, however, handles this
515 complexity without making additional demands on the user.

516
517 As a second example, we designed an MPRA library representing various arrange-
518 ments of binding sites for three liver-enriched transcription factors—HNF4A, PPARA,
519 and XBP1 (Fig. 3A,B). Following the oligo layout of Melnikov et al. [3], we start
520 by creating a Pool representing a template sequence that contains a putatively inert
521 100 bp cis-regulatory element (CRE) region (<cre>) and a barcode region (<bc>).
522 Next, we create a Pool representing the binding site for HNF4A. This Pool is then
523 passed to the `flip` Operation, which yields both orientations of the site (forward
524 and reverse complement). Pools for PPARA and XBP1 are created similarly. The
525 template Pool and three TFBS Pools are then passed to the `insertion_multiscan`
526 Operation, which samples 1,000 random placements of the TFBSs within the <cre>
527 region without overlap. Because `flip` runs in sequential mode, all $2^3 = 8$ orientation
528 combinations are generated for each placement, yielding 8,000 unique CRE variants.
529 Each sequence is then replicated three times using the `repeat` Operation. Finally, the
530 `get_barcodes` Operation is used to generate barcodes that are inserted into the <bc>
531 region of the sequence. The result is a library of 24,000 sequences (Fig. 3A).

532
533 Colored text helps users inspect the architecture of the resulting sequences. After
534 the template Pool is created, the `stylize` Operation is used to shade the <cre> region
535 gray. Each TFBS Pool is also assigned a distinct color when it is created (HNF4A
536 blue, PPARA purple, XBP1 orange); this is specified by the `style` parameter passed
537

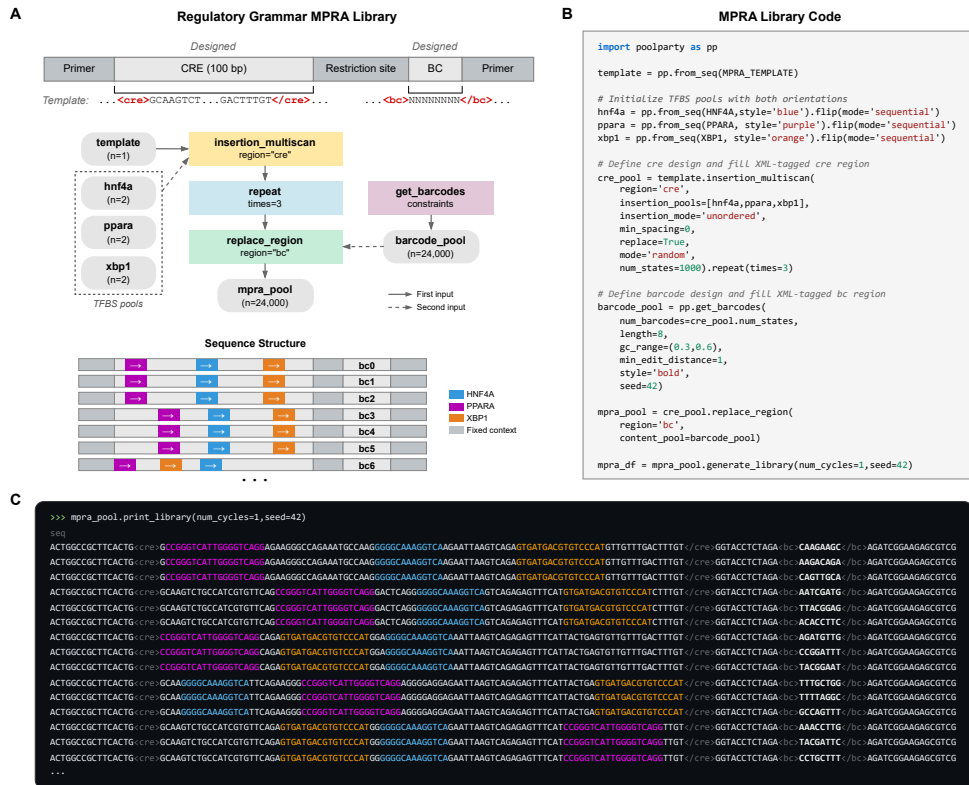


Fig. 3 A massively parallel reporter assay (MPRA) library for probing regulatory grammar. **(A)** Library schematic showing TFBS placements across a 100-bp CRE in representative variants, each linked to three different barcodes. **(B)** Python code for library construction. TFBS Pools are initialized with forward sequences and augmented with the `flip` Operation to include both orientations. **(C)** Output of `print_library`. TFBSs are colored by identity (HNF4A blue, PPARA purple, XBP1 orange); barcodes are shown in bold.

to `from_seq`. Barcodes are similarly rendered in bold text (Fig. 3B). These styles compose as sequences pass through the DAG, so that when `print_library` is called (Fig. 3C), the position, order, and orientation of each binding site are immediately visible.

3.3 Example 3: Surrogate modeling of SpliceAI predictions

Designed sequence libraries are also used to study genomic AI models [31–33, 35]. First, an in silico experiment is carried out in which the AI model scores all sequences

599 in a library. The results are then analyzed to characterize model behavior. One useful
600 way of analyzing these synthetic data is *surrogate modeling*, in which a mathematically
601 simple model—one whose structure and parameters are directly interpretable—is fit
602 to the predictions.
603

604
605 In addition to streamlining the design of sequence libraries, PoolParty facilitates in
606 silico experiments on genomic AI models in two ways. First, because sequence design
607 is separated from sequence generation, new sequences can be generated on demand as
608 the experiment proceeds. Second, the design card that PoolParty returns with each
609 sequence provides metadata that can be used directly as covariates in downstream
610 analyses, including surrogate modeling.
611

612
613 As a third example, we carried out a surrogate modeling study of SpliceAI [44],
614 a deep learning model of mRNA splicing. We asked how the insertion of a cryptic
615 5'ss near an existing (canonical) 5'ss affects the model's prediction for canonical site
616 usage. Using the 5'ss of SMN2 exon 7 as the canonical site, we used PoolParty to
617 construct a library in which cryptic 5'ss 9-mers of varying strength were inserted at
618 varying positions on either side of the canonical 5'ss (Methods). The specific values
619 for cryptic site strength and position were recorded in the design cards (Fig. 4A).
620

621
622 Each variant was paired with a matched control in which the cryptic site
623 was disrupted, and the effect of each insertion was quantified as the difference in
624 logit-transformed SpliceAI-predicted probabilities at the canonical site (Δlogit). Gen-
625 eralized additive models (GAMs) were used to separately model the effects of cryptic
626 site insertion at exonic and intronic positions. Each of these two models has three
627 terms, corresponding to the effects of position alone, strength alone, and interactions
628 between these covariates (Fig. 4A).
629

630
631 The GAMs closely reproduced the observed data (Fig. 4B,C), with held-out R^2 of
632 0.82 for exonic and 0.71 for intronic insertions (Fig. 4G). Cryptic sites on the exonic
633 side suppressed canonical site usage more strongly than those on the intronic side
634
644

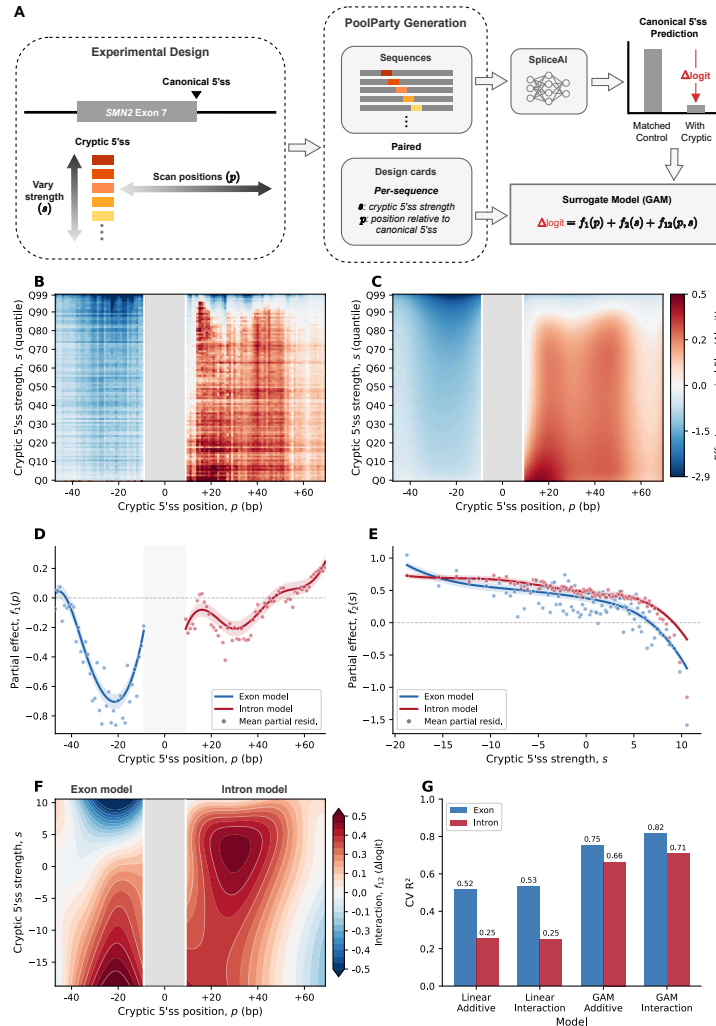


Fig. 4 Surrogate modeling of SpliceAI predictions. (A) Experimental design. Cryptic 5'ss 9-mers of varying MaxEntScan strength (s) are inserted at varying positions (p) flanking the canonical 5'ss of SMN2 exon 7. Each variant is paired with a control in which the splice-site GT dinucleotide is disrupted (GT→GA). PoolParty records s and p in design cards. The effect of each insertion on SpliceAI predictions at the canonical 5'ss is quantified as $\Delta\logit$ and modeled as a function of s and p using a generalized additive model (GAM). (B) Mean $\Delta\logit$ across the strength × position grid (100 × 100; each cell averages 20 distinct 9-mers). The grey band marks the canonical 5'ss region. (C) GAM predictions (including interaction term), using the same color scale as in B. (D) Partial dependence of the position term $f_1(p)$, shown separately for exon (blue) and intron (red) models. Shaded regions indicate 95% confidence intervals; points denote mean partial residuals. (E) Partial dependence of the strength term $f_2(s)$, with conventions as in D. (F) Interaction surface $f_{12}(p, s)$ for exon and intron models. (G) Block cross-validation R^2 for linear and GAM models, with and without interaction terms.

(Fig. 4D), with the effect peaking approximately 20 bp upstream of the canonical 5'ss. This is consistent with the observation that cryptic 5'ss are preferentially depleted near annotated ones on the exonic side [57]. Stronger cryptic sites drove greater suppression in both models (Fig. 4E). The effects of position and strength exhibited substantial interactions (Fig. 4F). Through a comparison to analogous linear models, we observed that the non-linear terms and interaction terms in the GAMs substantially improved predictive performance (Fig. 4G).

4 Discussion

We have presented PoolParty, a Python package that replaces the ad hoc scripting commonly used to design oligonucleotide libraries with a declarative, composable, and flexible framework. We demonstrated PoolParty through three use cases: a DMS library for protein GB1; an MPRA library probing transcriptional regulatory grammar; and a surrogate modeling analysis of the effects of cryptic 5'ss on SpliceAI predictions. These examples illustrate the flexibility of PoolParty and its applicability to both experimental and computational studies.

PoolParty was inspired by deep learning frameworks like PyTorch and TensorFlow, in which a high-level API is used to construct a DAG representing the desired neural network. The DAG then enables computation of model gradients via backpropagation. Similarly, in PoolParty, a backward pass through the DAG is used to decompose the state for a requested sequence into the specific Operation internal states needed to construct that sequence during a subsequent forward pass. This state-decomposition mechanism is a core innovation of PoolParty; it is what allows users to design complex oligo pools and test them before generating the full sequence library.

We also highlight PoolParty's design cards. Design cards allow the parameters governing each sequence's construction to serve directly as covariates in downstream analyses, eliminating the need for post hoc sequence parsing. In the SpliceAI example,

the cryptic splice-site strength and position recorded in each design card provided
the covariates used for surrogate modeling. We expect design cards to be increasingly
useful as in silico experiments on genomic AI models become more common.

In contrast to assay-specific tools such as VaLiAnT [41] and MPRAator [42],
PoolParty provides a single framework that supports DMS libraries, MPRA libraries,
and library designs that combine elements of both. Its DAG-based architecture lets
users freely compose arbitrary sequence generation and transformation operations.
Because the DAG encodes the full construction logic, structured design-card metadata
are generated automatically as a byproduct.

PoolParty was designed to address a focused computational problem: the stream-
lined design of DNA sequence libraries. As such, it has several limitations. PoolParty
is not a sequence optimization tool, e.g., for optimizing codon usage [58], satisfying
synthesis constraints on individual sequences [59], or designing sequences guided by
machine learning models [60]. Nor does it address the problem of physically construct-
ing DNA sequence libraries, e.g., by designing mutagenic primers [61]. We expect that
PoolParty can in many cases be productively paired with such complementary tools.

5 Conclusions

PoolParty should substantially ease the design of DMS experiments, MPRAAs, and
other multiplex assays of variant effect. As genomic AI models continue to advance,
PoolParty will also help researchers systematically probe and interpret the behavior
of these models. Moreover, because PoolParty’s composable design makes it straight-
forward to define new Operations, the framework can readily be extended to support
novel assay types and analysis strategies as they emerge.

Availability and requirements

Project name: PoolParty

783 **Project home page:** <https://github.com/jbkinney/poolparty-statetracker>
784
785 **Documentation:** <https://poolparty.readthedocs.io>
786 **Archived version:** DOI [10.5281/zenodo.19445098](https://doi.org/10.5281/zenodo.19445098)
787
788 **Operating system(s):** Platform independent
789
790 **Programming language:** Python (≥ 3.10)
791 **Other requirements:** NumPy (≥ 1.20), pandas (≥ 1.3), beartype ($\geq 0.22.9$),
792 pyfaidx ($\geq 0.8.1$), typing_extensions (≥ 4.0). The companion **statetracker** package
793 is installed automatically alongside PoolParty.
794
795
796 **License:** MIT License
797
798 **Any restrictions to use by non-academics:** None
799

800 801 **List of abbreviations**

802
803 **5'ss** 5' splice site
804
805 **CRE** Cis-regulatory element
806
807 **DAG** Directed acyclic graph
808
809 **DMS** Deep mutational scanning
810
811 **GAM** Generalized additive model
812
813 **MAVE** Multiplex assay of variant effect
814
815 **MPRA** Massively parallel reporter assay
816
817 **PWM** Position weight matrix
818
819 **TFBS** Transcription factor binding site
820

820 **Declarations**

821
822
823 **Ethics approval and consent to participate**
824

825 Not applicable.
826
827
828

Consent for publication	829
	830
Not applicable.	831
	832
	833
Availability of data and materials	834
	835
PoolParty is available on GitHub at https://github.com/jbkinney/poolparty-statetracker . The version described in this article is archived on Zenodo	836
at DOI 10.5281/zenodo.19445098. Code to reproduce the analyses in this paper is	837
included in the same repository.	838
	839
	840
	841
	842
	843
Competing interests	844
	845
The authors declare that they have no competing interests.	846
	847
	848
Funding	849
	850
This work was supported by the National Institutes of Health [R01HG011787].	851
Computational equipment was supported by the National Institutes of Health	852
[S10OD028632].	853
	854
	855
	856
	857
Authors' contributions	858
	859
Z.L. conceived the project. Z.L. and J.B.K. designed the project, wrote the software,	860
performed the analyses, and wrote the manuscript. Z.L. and A.C. wrote the docu-	861
mentation, developed the test suite, and deployed the software. J.B.K. supervised the	862
research. All authors read and approved the final manuscript.	863
	864
	865
	866
	867
Acknowledgements	868
	869
We thank Jack Desmarais, Evan Seitz, and Kai Loell for helpful discussions.	870
	871
	872
	873
	874

References

- [1] Patwardhan RP, Lee C, Litvin O, Young DL, Pe'er D, Shendure J. High-resolution analysis of DNA regulatory elements by synthetic saturation mutagenesis. *Nat Biotechnol.* 2009 12;27(12):1173–1175. <https://doi.org/10.1038/nbt.1589>.
- [2] Kinney JB, Murugan A, Callan CG, Cox EC. Using deep sequencing to characterize the biophysical mechanism of a transcriptional regulatory sequence. *Proc Natl Acad Sci USA.* 2010 05;107(20):9158–9163. <https://doi.org/10.1073/pnas.1004290107>.
- [3] Melnikov A, Murugan A, Zhang X, Tesileanu T, Wang L, Rogov P, et al. Systematic dissection and optimization of inducible enhancers in human cells using a massively parallel reporter assay. *Nat Biotechnol.* 2012 02;30(3):271–277. <https://doi.org/10.1038/nbt.2137>.
- [4] Kwasnieski JC, Mogno I, Myers CA, Corbo JC, Cohen BA. Complex effects of nucleotide variants in a mammalian cis-regulatory element. *Proc Natl Acad Sci USA.* 2012 11;109(47):19498–19503. <https://doi.org/10.1073/pnas.1210678109>.
- [5] Sharon E, Kalma Y, Sharp A, Raveh-Sadka T, Levo M, Zeevi D, et al. Inferring gene regulatory logic from high-throughput measurements of thousands of systematically designed promoters. *Nat Biotechnol.* 2012 05;30(6):521–530. <https://doi.org/10.1038/nbt.2205>.
- [6] Mogno I, Kwasnieski JC, Cohen BA. Massively parallel synthetic promoter assays reveal the in vivo effects of binding site variants. *Genome Res.* 2013;23(11):1908–1915. <https://doi.org/10.1101/gr.157891.113>.

[7] Levo M, Segal E. In pursuit of design principles of regulatory sequences. *Nat Rev Genet.* 2014;15(7):453–468. <https://doi.org/10.1038/nrg3684>.

[8] Inoue F, Ahituv N. Decoding enhancers using massively parallel reporter assays. *Genomics.* 2015;106(3):159–164. <https://doi.org/10.1016/j.ygeno.2015.06.005>.

[9] Vvedenskaya IO, Zhang Y, Goldman SR, Valenti A, Visone V, Taylor DM, et al. Massively systematic transcript end readout, "MASTER": transcription start site selection, transcriptional slippage, and transcript yields. *Mol Cell.* 2015;12;60(6):953–965. <https://doi.org/10.1016/j.molcel.2015.10.029>.

[10] Urtecho G, Tripp AD, Insigne KD, Kim H, Kosuri S. Systematic Dissection of Sequence Elements Controlling $\sigma 70$ Promoters Using a Genomically Encoded Multiplexed Reporter Assay in *Escherichia coli*. *Biochemistry.* 2019;03;58(11):1539–1551. <https://doi.org/10.1021/acs.biochem.7b01069>.

[11] Klein JC, Agarwal V, Inoue F, Keith A, Martin B, Kircher M, et al. A systematic evaluation of the design and context dependencies of massively parallel reporter assays. *Nat Methods.* 2020;17(11):1083–1091. <https://doi.org/10.1038/s41592-020-0965-y>.

[12] Romero IG, Lea AJ. Leveraging massively parallel reporter assays for evolutionary questions. *Genome Biol.* 2023;24(1):26. <https://doi.org/10.1186/s13059-023-02856-6>.

[13] La Fleur A, Shi Y, Seelig G. Decoding biology with massively parallel reporter assays and machine learning. *Genes Dev.* 2024;10;38(17-20):843–865. <https://doi.org/10.1101/gad.351800.124>.

[14] Fowler DM, Araya CL, Fleishman SJ, Kellogg EH, Stephany JJ, Baker D, et al. High-resolution mapping of protein sequence-function relationships. *Nat*

Methods. 2010 09;7(9):741–746. <https://doi.org/10.1038/nmeth.1492>.

[15] Hietpas RT, Jensen JD, Bolon DNA. Experimental illumination of a fitness landscape. *Proc Natl Acad Sci USA*. 2011 05;108(19):7896–7901. <https://doi.org/10.1073/pnas.1016024108>.

[16] Fowler DM, Fields S. Deep mutational scanning: a new style of protein science. *Nat Methods*. 2014;11(8):801–807. <https://doi.org/10.1038/nmeth.3027>.

[17] Olson CA, Wu NC, Sun R. A comprehensive biophysical description of pairwise epistasis throughout an entire protein domain. *Curr Biol*. 2014 11;24(22):2643–2651. <https://doi.org/10.1016/j.cub.2014.09.072>.

[18] Adams RM, Mora T, Walczak AM, Kinney JB. Measuring the sequence-affinity landscape of antibodies with massively parallel titration curves. *eLife*. 2016 12;5:e23156. <https://doi.org/10.7554/elife.23156>.

[19] Findlay GM, Daza RM, Martin B, Zhang MD, Leith AP, Gasperini M, et al. Accurate classification of BRCA1 variants with saturation genome editing. *Nature*. 2018;562(7726):217–222. <https://doi.org/10.1038/s41586-018-0461-z>.

[20] Starr TN, Greaney AJ, Hilton SK, Ellis D, Crawford KHD, Dingens AS, et al. Deep mutational scanning of SARS-CoV-2 receptor binding domain reveals constraints on folding and ACE2 binding. *Cell*. 2020;182(5):1295–1310.e20. <https://doi.org/10.1016/j.cell.2020.08.012>.

[21] Berger M, Philippakis A, Qureshi A, He F, Estep P, Bulyk M. Compact, universal DNA microarrays to comprehensively determine transcription-factor binding site specificities. *Nat Biotechnol*. 2006 11;24(11):1429–1435. <https://doi.org/10.1038/nbt1246>.

[22] Maerkl SJ, Quake SR. A systems approach to measuring the binding energy landscapes of transcription factors. *Science*. 2007 01;315(5809):233–237. <https://doi.org/10.1126/science.1131007>.

[23] Noyes M, Christensen R, Wakabayashi A, Stormo G, Brodsky M, Wolfe S. Analysis of homeodomain specificities allows the family-wide prediction of preferred recognition sites. *Cell*. 2008 06;133(7):1277–1289. <https://doi.org/10.1016/j.cell.2008.05.023>.

[24] Gordân R, Shen N, Dror I, Zhou T, Horton J, Rohs R, et al. Genomic Regions Flanking E-Box Binding Sites Influence DNA Binding Specificity of bHLH Transcription Factors through DNA Shape. *Cell Rep*. 2013;3(4):1093–1104. <https://doi.org/10.1016/j.celrep.2013.03.014>.

[25] Le DD, Shimko TC, Aditham AK, Keys AM, Longwell SA, Orenstein Y, et al. Comprehensive, high-resolution binding energy landscapes reveal context dependencies of transcription factor binding. *Proc Natl Acad Sci USA*. 2018 04;115(16):E3702–E3711. <https://doi.org/10.1073/pnas.1715888115>.

[26] Shen N, Zhao J, Schipper JL, Zhang Y, Bepler T, Leehr D, et al. Divergence in DNA Specificity among Paralogous Transcription Factors Contributes to Their Differential In Vivo Binding. *Cell Syst*. 2018 04;6(4):470–483.e8. <https://doi.org/10.1016/j.cels.2018.02.009>.

[27] Khetan S, Carroll BS, Bulyk ML. Multiple overlapping binding sites determine transcription factor occupancy. *Nature*. 2025;646(8086):1001–1011. <https://doi.org/10.1038/s41586-025-09472-3>.

[28] Kinney JB, McCandlish DM. Massively parallel assays and quantitative sequence-function relationships. *Annu Rev Genomics Hum Genet*. 2019 08;20(1):99–127.

1059 <https://doi.org/10.1146/annurev-genom-083118-014845>.
1060
1061 [29] Starita LM, Ahituv N, Dunham MJ, Kitzman JO, Roth FP, Seelig G, et al.
1062 Variant Interpretation: Functional Assays to the Rescue. *Am J Hum Genet*. 2017
1063 09;101(3):315–325. <https://doi.org/10.1016/j.ajhg.2017.07.014>.
1064
1065
1066
1067 [30] Weile J, Roth FP. Multiplexed assays of variant effects contribute to a growing
1068 genotype-phenotype atlas. *Hum Genet*. 2018 09;137(9):665–678. [https://doi.org/](https://doi.org/10.1007/s00439-018-1916-x)
1069 [10.1007/s00439-018-1916-x](https://doi.org/10.1007/s00439-018-1916-x).
1070
1071
1072
1073 [31] Avsec Ž, Weilert M, Shrikumar A, Krueger S, Alexandari A, Dalal K, et al. Base-
1074 Resolution Models of Transcription-Factor Binding Reveal Soft Motif Syntax. *Nat*
1075 *Genet*. 2021 Mar;53(3):354–366. <https://doi.org/10.1038/s41588-021-00782-6>.
1076
1077
1078
1079 [32] Toneyan S, Koo PK. Interpreting Cis-Regulatory Interactions from Large-Scale
1080 Deep Neural Networks. *Nat Genet*. 2024 Nov;56(11):2517–2527. [https://doi.org/](https://doi.org/10.1038/s41588-024-01923-3)
1081 [10.1038/s41588-024-01923-3](https://doi.org/10.1038/s41588-024-01923-3).
1082
1083
1084
1085 [33] Seitz EE, McCandlish DM, Kinney JB, Koo PK. Interpreting Cis-Regulatory
1086 Mechanisms from Genomic Deep Neural Networks Using Surrogate Mod-
1087 els. *Nat Mach Intell*. 2024 Jun;6(6):701–713. [https://doi.org/10.1038/](https://doi.org/10.1038/s42256-024-00851-5)
1088 [s42256-024-00851-5](https://doi.org/10.1038/s42256-024-00851-5).
1089
1090
1091
1092 [34] Seitz EE, McCandlish DM, Kinney JB, Koo PK. Uncovering the Mecha-
1093 nistic Landscape of Regulatory DNA with Deep Learning. *bioRxiv*. 2025;p.
1094 2025.10.07.681052. <https://doi.org/10.1101/2025.10.07.681052>.
1095
1096
1097
1098 [35] Nagai M, Murphy AE, Rizzo K, Koo PK. Toward Interpretable and Generalizable
1099 AI in Regulatory Genomics; 2026. ArXiv:2602.01230.
1100
1101
1102
1103
1104

[36] Lee NK, Tang Z, Toneyan S, Koo PK. EvoAug: improving generalization and interpretability of genomic deep neural networks with evolution-inspired data augmentations. *Genome Biol.* 2023;24(1):105. <https://doi.org/10.1186/s13059-023-02941-w>.

[37] Cock PJA, Antao T, Chang JT, Chapman BA, Cox CJ, Dalke A, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics.* 2009;25(11):1422–1423. <https://doi.org/10.1093/bioinformatics/btp163>.

[38] Pereira F, Azevedo F, Carvalho Â, Ribeiro GF, Budde MW, Johansson B. Pydna: a simulation and documentation tool for DNA assembly strategies using python. *BMC Bioinformatics.* 2015;16(1):142. <https://doi.org/10.1186/s12859-015-0544-x>.

[39] Klie A, Laub D. SeqPro (Sequence processing toolkit); 2023. Software. Available from: <https://github.com/ML4GLand/SeqPro>.

[40] Schreiber J. tangermeme: A toolkit for understanding cis-regulatory logic using deep learning models. *bioRxiv.* 2025;p. 2025.08.08.669296. <https://doi.org/10.1101/2025.08.08.669296>.

[41] Barbon L, Offord V, Radford EJ, Butler AP, Gerety SS, Adams DJ, et al. Variant Library Annotation Tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments. *Bioinformatics.* 2022;38(4):892–899. <https://doi.org/10.1093/bioinformatics/btab776>.

1151 [42] Georgakopoulos-Soares I, Jain N, Gray JM, Hemberg M. MPRAnator: a web-
1152 based tool for the design of massively parallel reporter assay experiments. *Bioin-*
1153 formatics. 2017;33(1):137–138. <https://doi.org/10.1093/bioinformatics/btw584>.
1154
1155
1156
1157 [43] Ghazi AR, Chen ES, Henke DM, Madan N, Edelstein LC, Shaw CA. Design
1158 tools for MPRA experiments. *Bioinformatics*. 2018;34(15):2682–2683. [https:](https://doi.org/10.1093/bioinformatics/bty150)
1159 [//doi.org/10.1093/bioinformatics/bty150](https://doi.org/10.1093/bioinformatics/bty150).
1160
1161
1162
1163 [44] Jaganathan K, Panagiotopoulou SK, McRae JF, Darbandi SF, Knowles D, Li
1164 YI, et al. Predicting Splicing from Primary Sequence with Deep Learning. *Cell*.
1165 2019 Jan;176(3):535–548.e24. <https://doi.org/10.1016/j.cell.2018.12.015>.
1166
1167
1168
1169 [45] Wong MS, Kinney JB, Krainer AR. Quantitative Activity Profile and Context
1170 Dependence of All Human 5' Splice Sites. *Mol Cell*. 2018 Sep;71(6):1012–1026.e3.
1171 <https://doi.org/10.1016/j.molcel.2018.07.033>.
1172
1173
1174 [46] Yeo G, Burge CB. Maximum Entropy Modeling of Short Sequence Motifs with
1175 Applications to RNA Splicing Signals. *J Comput Biol*. 2004 Mar;11(2-3):377–394.
1176 <https://doi.org/10.1089/1066527041410418>.
1177
1178
1179
1180 [47] Servén D, Brummitt C. pyGAM: Generalized Additive Models in Python; 2018.
1181 Software (Zenodo). Available from: <https://doi.org/10.5281/zenodo.1208723>.
1182
1183
1184 [48] Otwinowski J. Biophysical Inference of Epistasis and the Effects of Mutations
1185 on Protein Stability and Function. *Mol Biol Evol*. 2018 10;35(10):2345–2354.
1186 <https://doi.org/10.1093/molbev/msy141>.
1187
1188
1189
1190 [49] Otwinowski J, McCandlish DM, Plotkin JB. Inferring the shape of global epis-
1191 tasis. *Proc Natl Acad Sci USA*. 2018 08;115(32):E7550–E7558. [https://doi.org/](https://doi.org/10.1073/pnas.1804015115)
1192 [10.1073/pnas.1804015115](https://doi.org/10.1073/pnas.1804015115).
1193
1194
1195
1196

[50] Tareen A, Kooshkbaghi M, Posfai A, Ireland WT, McCandlish DM, Kinney JB. MAVE-NN: learning genotype-phenotype maps from multiplex assays of variant effect. *Genome Biol.* 2022;23(1):98. <https://doi.org/10.1186/s13059-022-02661-7>.

[51] Faure AJ, Lehner B. MoCHI: Neural Networks to Fit Interpretable Models and Quantify Energies, Energetic Couplings, Epistasis, and Allostery from Deep Mutational Scanning Data. *Genome Biol.* 2024 Dec;25(1):303. <https://doi.org/10.1186/s13059-024-03444-y>.

[52] King DM, Hong CKY, Shepherdson JL, Granas DM, Maricque BB, Cohen B. Synthetic and genomic regulatory elements reveal aspects of cis-regulatory grammar in Mouse Embryonic Stem Cells. *eLife.* 2020;9:e41279. <https://doi.org/10.7554/elife.41279>.

[53] Smith RP, Taher L, Patwardhan RP, Kim MJ, Inoue F, Shendure J, et al. Massively parallel decoding of mammalian regulatory sequences supports a flexible organizational model. *Nat Genet.* 2013;45(9):1021–1028. <https://doi.org/10.1038/ng.2713>.

[54] Grossman SR, Zhang X, Wang L, Engreitz J, Melnikov A, Rogov P, et al. Systematic dissection of genomic features determining transcription factor binding and enhancer function. *Proc Natl Acad Sci USA.* 2017;114(7):E1291–E1300. <https://doi.org/10.1073/pnas.1621150114>.

[55] Weingarten-Gabbay S, Nir R, Lubliner S, Sharon E, Kalma Y, Weinberger A, et al. Systematic interrogation of human promoters. *Genome Res.* 2019 02;29(2):171–183. <https://doi.org/10.1101/gr.236075.118>.

[56] Georgakopoulos-Soares I, Deng C, Agarwal V, Chan CSY, Zhao J, Inoue F, et al. Transcription Factor Binding Site Orientation and Order Are Major Drivers of

1243 Gene Regulatory Activity. Nat Commun. 2023 Apr;14(1):2333. [https://doi.org/](https://doi.org/10.1038/s41467-023-37960-5)
1244 [10.1038/s41467-023-37960-5](https://doi.org/10.1038/s41467-023-37960-5).
1245
1246
1247 [57] Dawes R, Joshi H, Cooper ST. Empirical Prediction of Variant-Activated Cryp-
1248 tic Splice Donors Using Population-Based RNA-Seq Data. Nat Commun. 2022
1249 Mar;13(1):1655. <https://doi.org/10.1038/s41467-022-29271-y>.
1250
1251
1252
1253 [58] Swainston N, Currin A, Green L, Breitling R, Day PJ, Kell DB. CodonGenie:
1254 optimised ambiguous codon design tools. PeerJ Comput Sci. 2017;3:e120. [https:](https://doi.org/10.7717/peerj-cs.120)
1255 [//doi.org/10.7717/peerj-cs.120](https://doi.org/10.7717/peerj-cs.120).
1256
1257
1258
1259 [59] Zulkower V, Rosser S. DNA Chisel, a versatile sequence optimizer. Bioinformat-
1260 ics. 2020;36(16):4508–4509. <https://doi.org/10.1093/bioinformatics/btaa558>.
1261
1262
1263 [60] Schreiber J, Lorbeer FK, Heinzl M, Lu YY, Stark A, Noble WS. Programmatic
1264 design and editing of cis-regulatory elements. bioRxiv. 2025;p. 2025.04.22.650035.
1265 <https://doi.org/10.1101/2025.04.22.650035>.
1266
1267
1268
1269 [61] Hiraga K, Mejzlik P, Marcisin M, Vostrosablin N, Gromek A, Arnold J, et al.
1270 Mutation Maker, An Open Source Oligo Design Platform for Protein Engineer-
1271 ing. ACS Synth Biol. 2021;10(2):357–370. [https://doi.org/10.1021/acssynbio.](https://doi.org/10.1021/acssynbio.0c00542)
1272 [0c00542](https://doi.org/10.1021/acssynbio.0c00542).
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288